Retrieval Augmented Generation (RAG)

Pawan Goyal
Department of Computer Science and Engineering
IIT Kharagpur
https://cse.iitkgp.ac.in/~pawang/

AI4ICPS IIT Kharagpur

When an LLM is left to guess

With no source to draw on, a model still answers — confidently. Four ways that goes wrong in practice.

Stale knowledge

"What's the current version of our SDK?"

The model answers with a version frozen at its training cutoff — months out of date.

Fabricated sources

"Cite a study that supports this."

The model invents a real-sounding paper, author and year that don't exist.

No access to your data

"What's our PTO carryover policy?"

The model guesses a generic policy — it has never seen your handbook.

Confident hallucination

"What does the merge_safe flag do?"

The model describes a plausible flag that isn't in the product at all.

Motivation for RAG

What LLMs don't know

Private Databases
LLMs can't access confidential information

Hard to access information
Some information isn't widely available online, making it inaccessible to LLMs

Real time data
LLMs are trained on past data and don't update automatically

How to make sure the LLM knows this useful information?

THE PROBLEM

How does RAG help?

A language model only knows what it saw during training. Ask about your documents, last week's news, or a private policy, and it guesses — often confidently and wrong.

LLM on its own

- Frozen at its training cutoff — no new or private data
- Invents facts, names and citations that sound right
- Can't tell you where an answer came from
- Re-training to add knowledge is slow and expensive

LLM + RAG

- Reads the latest and your private documents at ask-time
- Answers are grounded in retrieved, real passages
- Can cite the exact source it used
- Update knowledge by updating files — no re-training

BEFORE & AFTER

The same question, grounded with RAG

Hand the model the right passage and the confident guess becomes a sourced answer.

QUESTION	WITHOUT RAG	WITH RAG
What's our refund window?	"Returns are accepted within 14 days." ✗ *Plausible, but invented*	**"30 days with a receipt."** `returns_policy.md · §2`
Which SDK version is current?	"The latest is v3.2." ✗ *Frozen at training cutoff*	**"v5.1, released Aug 2026."** `changelog.md`
Does our plan cover MRI scans?	"Most plans cover MRIs with a referral." ✗ *Generic, not your plan*	**"Yes — 80% after deductible."** `benefits.pdf · §4.2`

AI4ICPS IIT Kharagpur

RAG = look it up, then write the answer

Instead of answering from memory, the system first fetches relevant text, then asks the model to answer using that text.

1 · Retrieval

Find the passages most likely to answer the question.

- Search a knowledge base (often a vector DB)
- Return the top few most relevant chunks
- The “facts” half — no writing yet

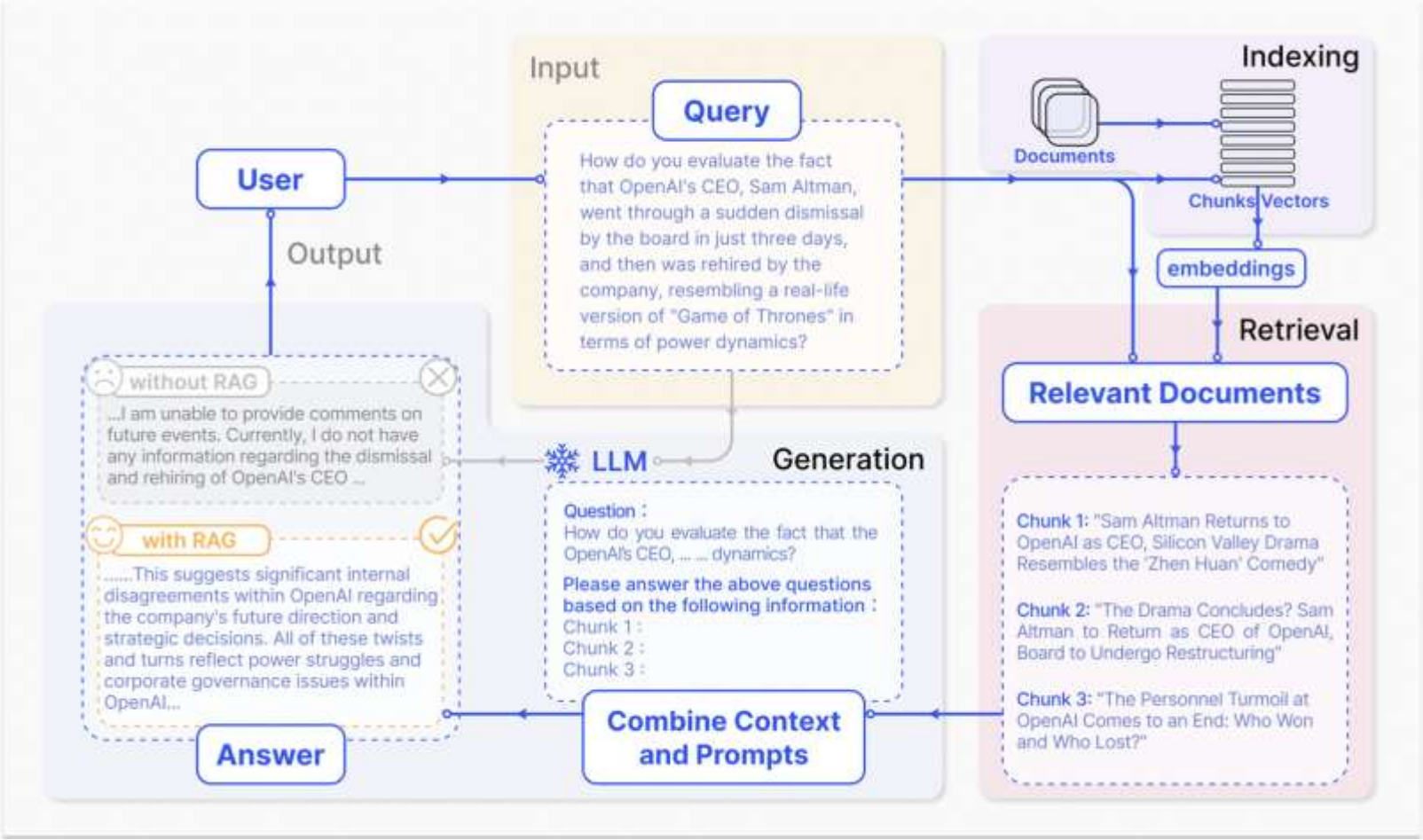
+

2 · Generation

Write a fluent answer, grounded in those passages.

- The retrieved text is added to the prompt
- The LLM answers using that context
- The “writing” half — fluent + grounded

AI4CPS IIT Kharagpur



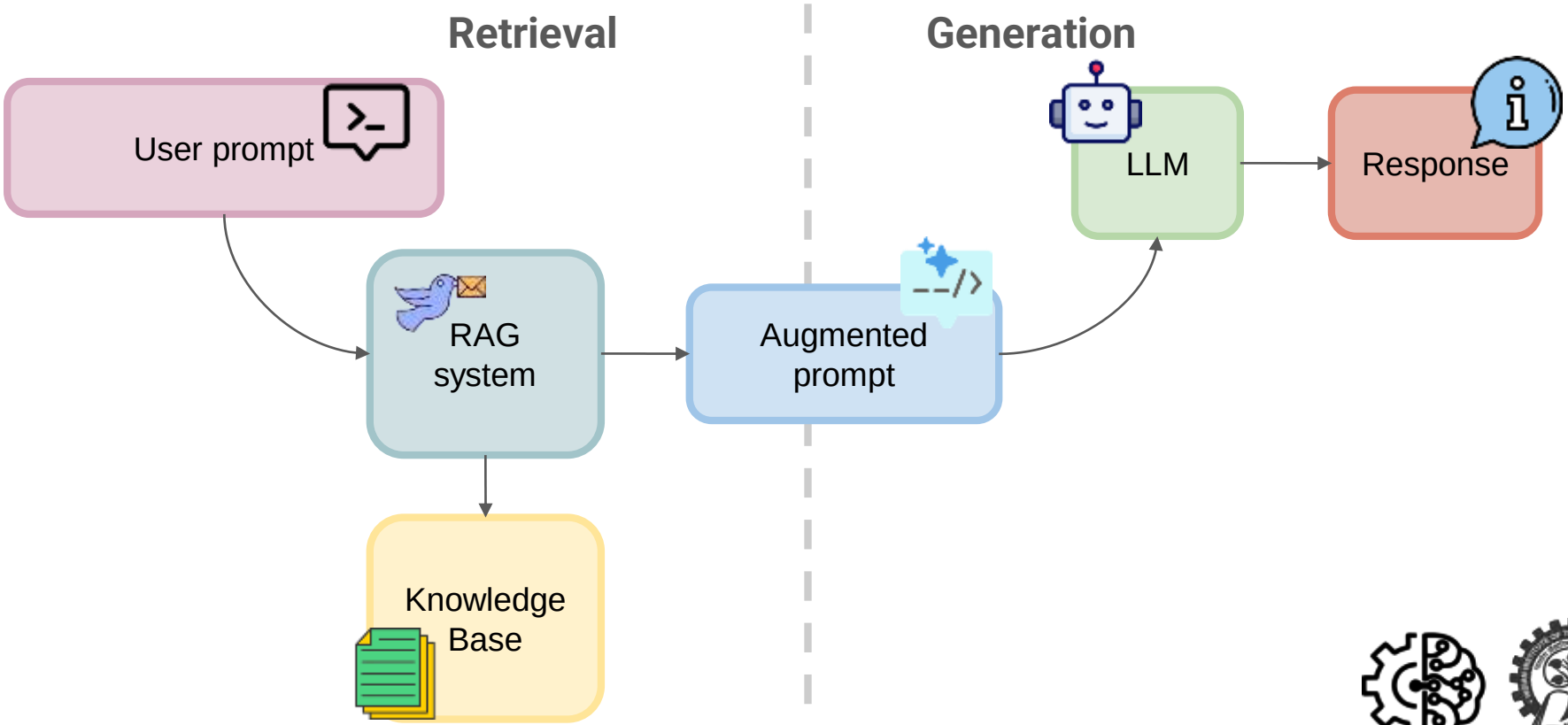
Retrieval Augmented Generation

Source: Retrieval-Augmented Generation for Large Language Models: A Survey

AI4ICPS

IIT Kharagpur

RAG architecture



AI4ICPS IIT Kharagpur

RAG: Retrieval

Goal: find a small subset of elements in a datastore that are the most similar to the query

sim: a similarity score between two pieces of text

Sparse vs. Dense Retrieval

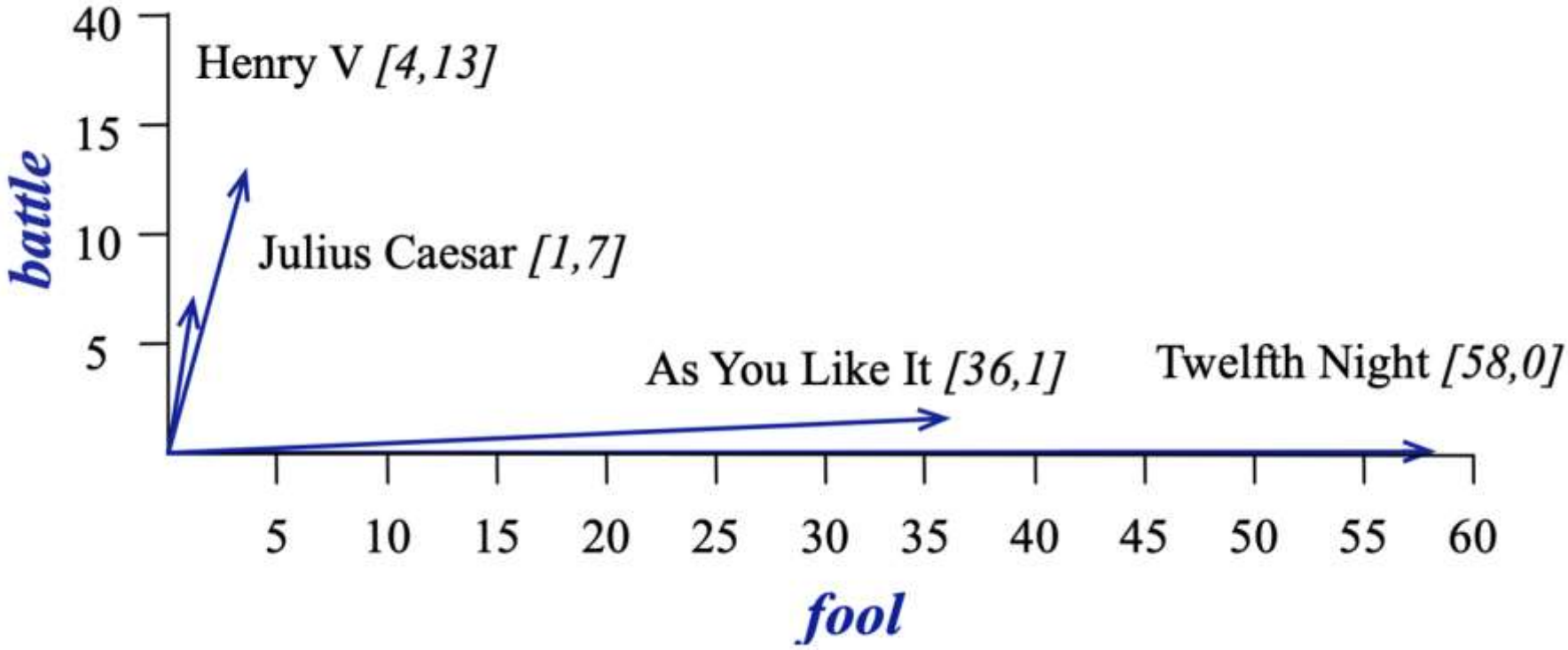
Sparse Retrieval

The vector space model of IR

Gerard Salton, 1971

Represent a document as a vector of counts of the words it contains.

Sparse Retrieval



TF-IDF: ranking keywords, in two questions

A word matters in a document when it's frequent HERE but rare EVERYWHERE else

TF

1. Term Frequency

How often does the word appear in THIS document?

Appears a lot → the doc is probably about it.

"insurance" ×3 here → strong signal

IDF

2. Inverse Document Frequency

How rare is the word across ALL documents?

Rare word → very informative. Common word → tells you nothing.

"the" is everywhere → ignore it

`Score = TF × IDF.` A word scores high only when it's frequent in the document AND rare in the collection.

Term Frequency (TF), by example

Just count how often each word shows up in the document

Document: *“car insurance auto insurance”*



“insurance” appears twice → highest TF. The document is most about insurance.

Inverse Document Frequency (IDF), by example

Across 1,000,000 documents — rarer words carry more signal

word	appears in… (out of 1,000,000 docs)	rarity weight (IDF)
the	~1,000,000	*everywhere → useless* 0.0
best	50,000	1.3
car	10,000	2.0
auto	5,000	2.3
insurance	1,000	*rare → most informative* 3.0

"insurance" is rare → high IDF. "the" is everywhere → ~zero. *IDF = log(total docs ÷ docs containing the word).*

AI4ICPS IIT Kharagpur

tf-idf weighting and Scoring

- The tf-idf weight of a term is the product of its tf weight and its idf weight.

$$w_{t,d} = (1 + \log_{10} \mathrm{tf}_{t,d}) \times \log_{10}(N / \mathrm{df}_t)$$

- One of the best known weighting scheme in sparse information retrieval
  - Alternative names: tf.idf, tf x idf
- Increases with the number of occurrences of term within a document
- Increases with the rarity of the term in the collection

$$\mathrm{Score}(q,d) = \sum_{t \in q \cap d} \mathrm{tf.idf}_{t,d}$$

tf-idf example

Document: *car insurance auto insurance*
Query: *best car insurance*

*We do not use idf with the document to avoid (idf)*idf*

Term	Query						Document				Prod
	tf-raw	tf-wt	df	idf	wt	n'lize	tf-raw	tf-wt	wt	n'lize	
auto	0	0	5000	2.3	0	0	1	1	1	0.52	0
best	1	1	50000	1.3	1.3	0.34	0	0	0	0	0
car	1	1	10000	2.0	2.0	0.52	1	1	1	0.52	0.27
insurance	1	1	1000	3.0	3.0	0.78	2	1.3	1.3	0.68	0.53

Doc length = $\sqrt{1^2 + 0^2 + 1^2 + 1.3^2} \approx 1.92$

Query length: 3.83

N = 1 million

Score = 0+0+0.27+0.53 = 0.8

AI4ICPS IIT Kharagpur

BM25: Another very commonly used scheme

The BM25 score for a document d given a query q is calculated as the sum of scores for each query term t that appears in the document:

$$\text{Score}(q,d) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{tf(t,d) \cdot (k_1 + 1)}{tf(t,d) + k_1 \cdot (1 - b + b \cdot \frac{|d|}{avgdl})}$$

$$\text{IDF}(t) = \log\left(\frac{N - df(t) + 0.5}{df(t) + 0.5}\right)$$

Problem with Sparse Retrieval

Sparse retrieval matches **strings**. Dense retrieval matches **meaning**.

QUERY

“How much vacation do I get?”

DOCUMENT

“Employees accrue 18 days of paid leave annually.”

BM25 / TF–IDF: zero shared words → score ≈ 0. The right answer is invisible.

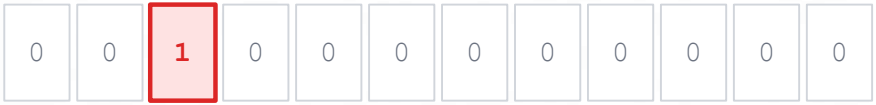
Dense embeddings: “vacation” and “paid leave” sit close in vector space → high similarity.

This vocabulary-mismatch gap is the whole reason dense retrieval exists.

Dense Representations

Replace one sparse index with a short, dense vector where every dimension carries meaning

SPARSE (One-Hot)



… (50,000 dims)

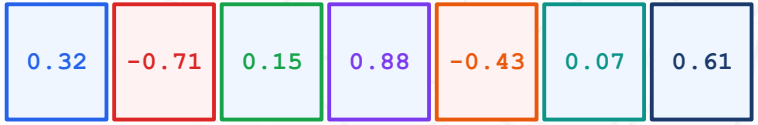
✗ 99%+ zeros

✗ No similarity

✗ Huge memory

learn weights →

DENSE (Word Embedding)



(300 dims — every value carries meaning)

✓ All values active

✓ Geometry = meaning

✓ Compact & fast

Geometry encodes meaning — famous examples:

```
vec("king") - vec("man") + vec("woman")   ≈ vec("queen")
vec("Paris") - vec("France") + vec("Germany")   ≈ vec("Berlin")
cosine( vec("cat"), vec("kitten") )   ≈ 0.92  ✓
```

Vector Embeddings

Words
word2vec · GloVe

"bank" — river or money?
static: one vector per word, context-blind



Sentences
SBERT · E5 · GTE

"How do I open an account?"
contrastively trained: paraphrases pulled together



Chunks
long-context embedders · late chunking

a paragraph of your document
the unit of retrieval in RAG — this is what we index



Anything
CLIP · CLAP · CodeBERT

images · audio · code · tables
one shared space: a photo of a dog = the word "dog"



AI4ICPS IIT Kharagpur

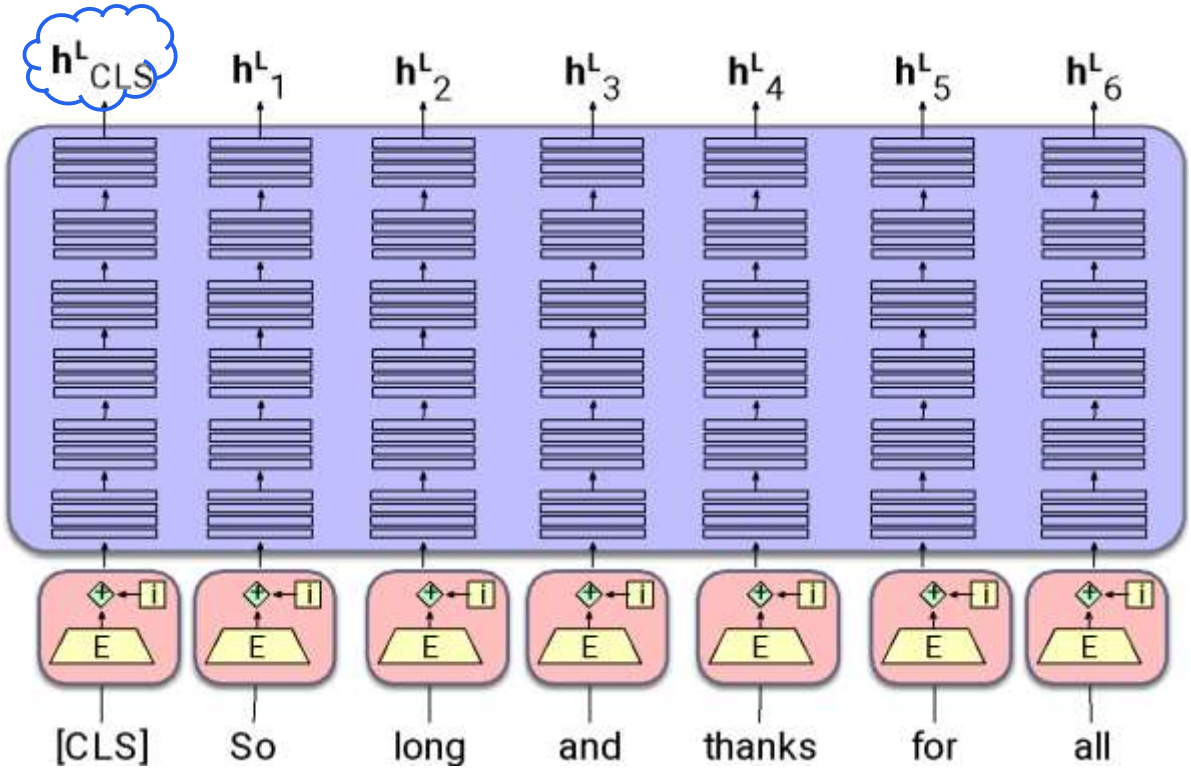
Hypothetical version of dense retrieval: static embeddings

Replace tf-idf vectors with, e.g., word2vec

- Query: the mean of the embeddings of each query word
- Doc: the mean of the doc word embeddings
- Now just compute query-doc cosine as normal.

We don't do this because **contextual embeddings** work much better!

How do we get sentence embeddings using BERT?

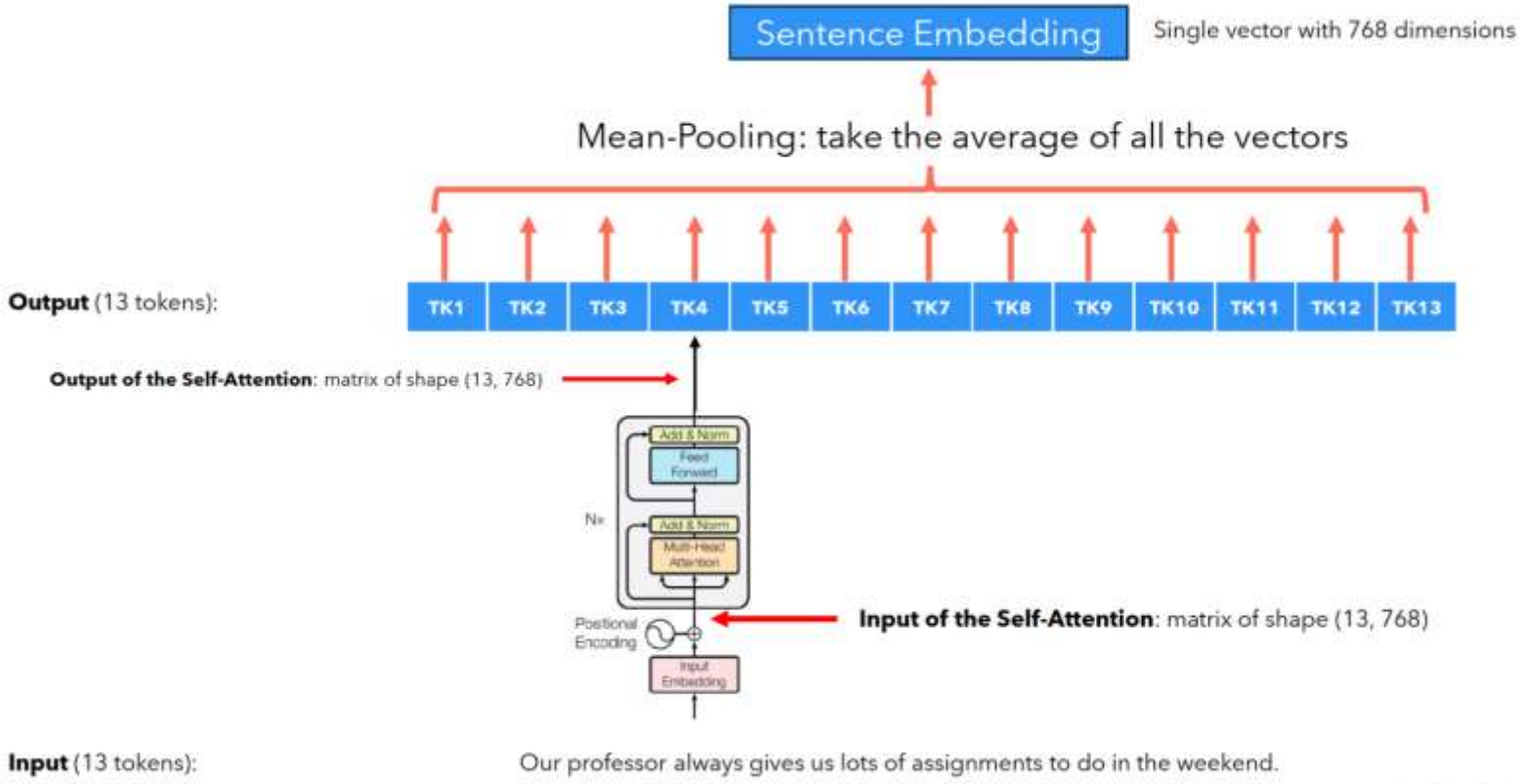


We can get sentence embeddings from BERT using the [CLS] token, which is basically the whole sentence representation

https://web.stanford.edu/~jurafsky/slp3/

AI4ICPS IIT Kharagpur

Sentence Embedding with BERT



Source: https://github.com/hkproj/retrieval-augmented-generation-notes

AI4ICPS

IIT Kharagpur

Sentence Embedding: Comparison

- How can we compare Sentence Embeddings to see if two sentences have similar "meaning"? We could use the cosine similarity, which measures the cosine of the angle between the two vectors. A small angle results in a high cosine similarity score.

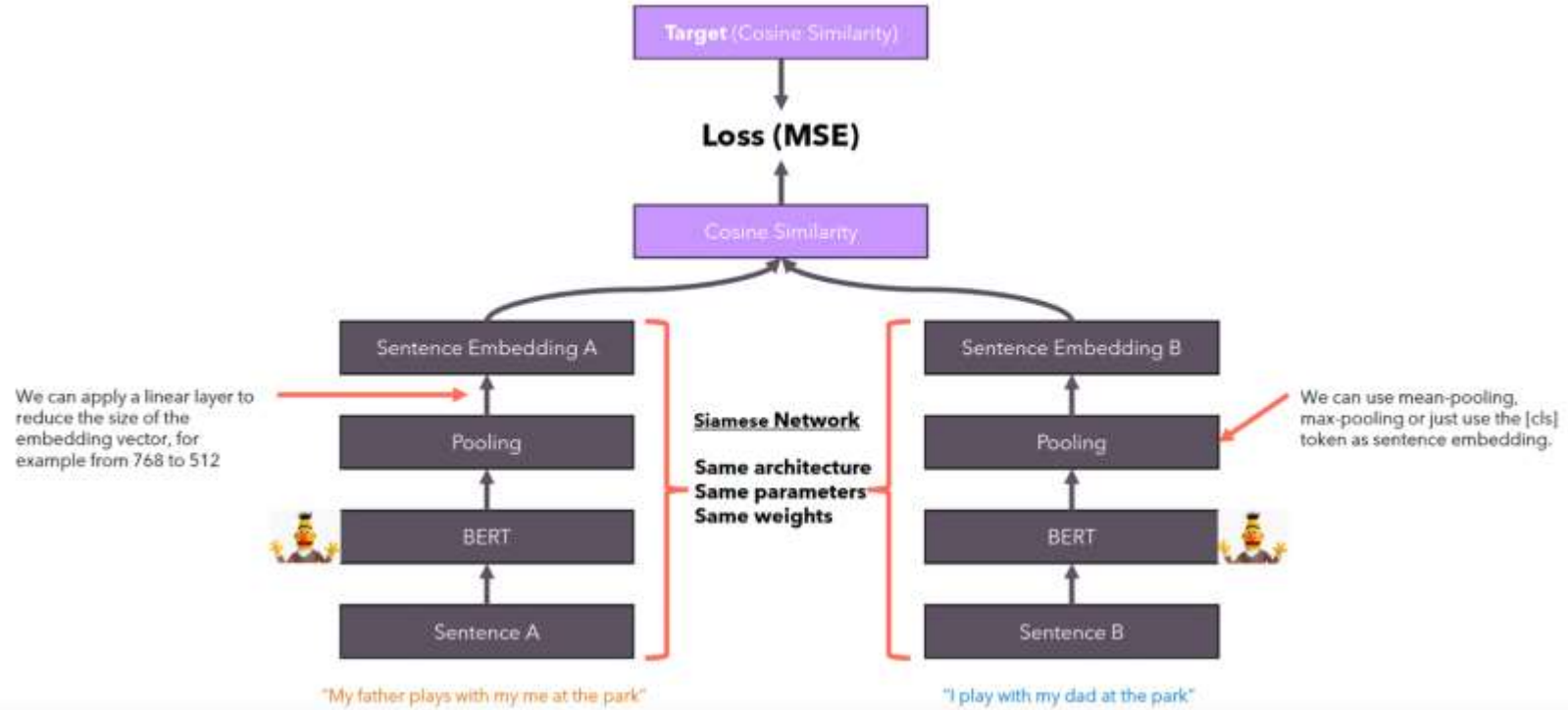
$$\text{cosine similarity} = S_C(A, B) := \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2} \cdot \sqrt{\sum_{i=1}^{n} B_i^2}}$$

- **But there's a problem**: nobody told BERT that the embeddings it produces should be comparable with the cosine similarity, that is, two similar sentences should be represented by vectors pointing to the same direction in space. How can we teach BERT to produce embeddings that can be compared with a similarity function of our choice?

Source: https://github.com/hkproj/retrieval-augmented-generation-notes

AI4ICPS IIT Kharagpur

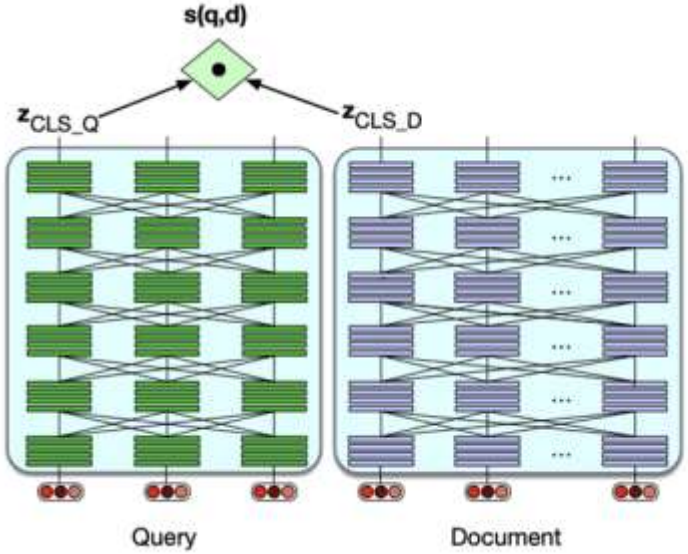
Sentence BERT: Architecture



Source: https://github.com/hkproj/retrieval-augmented-generation-notes

AI4ICPS IIT Kharagpur

Dense Retrieval: bi-encoder



Encode doc vectors in advance.

Encode query when it arrives

- Score is dot product between query vector and precomputed doc vector
- Cheaper but less accurate

Where dense search quietly fails

A real support-bot query — semantic search returns the wrong thing

Query: *“How do I fix error ERR_2043?”*

× Dense returns…

“General guidance on error handling and best practices…”

Semantically near “error” — but useless here.

✓ What you needed…

“ERR_2043: database connection timeout — increase the pool size.”

The exact match — but dense ranked it far down.

Why? A code like **ERR_2043** has no “meaning” to embed. To an embedding model, **ERR_2043** and **ERR_2049** look nearly identical — yet mean completely different things.

The fix: sparse (keyword) search

Old-school exact-word matching — exactly right for codes, names & rare terms

Sparse search matches the actual words. Search “ERR_2043” and it finds the passages that literally contain ERR_2043.

Perfect for the things dense search blurs:

Error codes	Product SKUs	Function names	Legal cites
`ERR_2043`	`MX-4471-B`	`calc_tax_v2()`	`§12(a)(3)`

But how does it decide which document is the best match? *That’s what TF-IDF answers →*

AI4ICPS

IIT Kharagpur

Dense vs Sparse: two blind spots

Each is strong exactly where the other is weak

DENSE (embeddings)

✓ **Catches meaning**
"vacation" ≈ "paid leave"

× **Misses exact rare tokens**
error codes, SKUs, function names, IDs

SPARSE (BM25 / TF-IDF)

✓ **Nails exact keywords**
rare tokens, codes, precise phrases

× **Blind to synonyms**
zero shared words → score ≈ 0

The two failure modes don't overlap — so what if we just run both?

Hybrid search: run both, fuse the results

Cover meaning AND exact terms — neither blind spot bites

Dense retriever
top matches by meaning

Sparse retriever (BM25)
top matches by exact words

→

FUSE
the rankings

(RRF — next slide)

→

One combined ranked list

When to use it: any time queries mix natural language with exact identifiers — code + logs, product catalogs, legal text, support tickets.
Practical note: adding hybrid is often a bigger win than upgrading to a fancier (and pricier) embedding model.

AI4ICPS IIT Kharagpur

Combining lists: Reciprocal Rank Fusion (RRF)

A dead-simple way to merge two ranked lists — using only the ranks

For each document, add up `1 / (k + rank)` from every list. (k ≈ 60) Higher total = better.

Dense list	Sparse list
1. Doc A	1. Doc C
2. Doc B	2. Doc A
3. Doc C	3. Doc B

Fused score (k = 60)

Doc A	1/61 + 1/62	0.0325	← wins
Doc C	1/63 + 1/61	0.0323	
Doc B	1/62 + 1/63	0.0320	

Doc A is high in BOTH lists → beats Doc C, which is #1 in only one.

Why it's nice: no need to make dense scores and BM25 scores comparable — you only use each doc's position. Consistent-in-both beats spiky-in-one.

Retrieve then rerank: the two-stage pattern

Retrieve a lot fast, then re-score the top few precisely

1 Retrieve

Hybrid (dense + sparse) pulls the top ~100 candidates from millions.

fast · approximate

→

2 Rerank

A slower, sharper model re-scores those ~100 and reorders them.

slow · precise

→

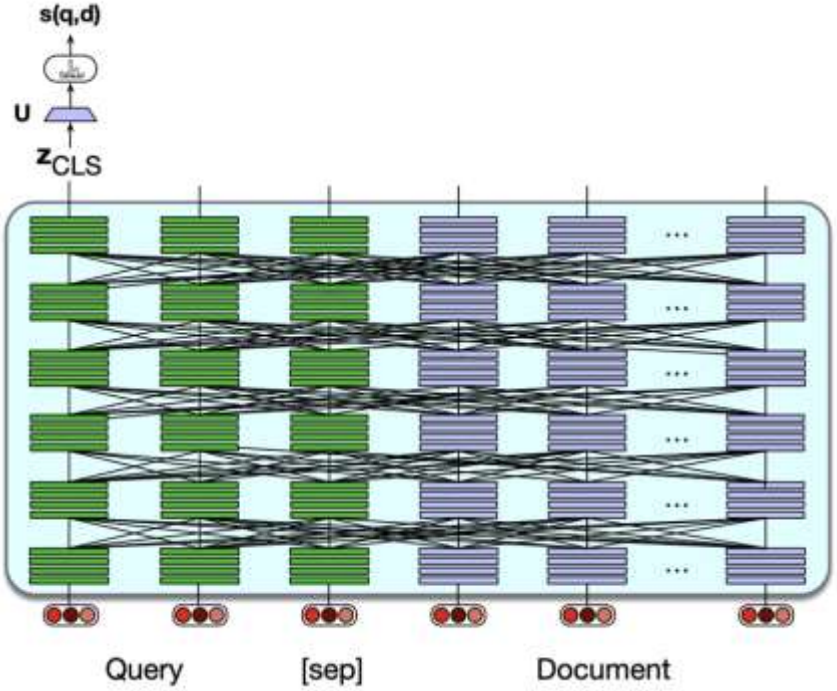
3 Generate

The best 5–20 chunks go to the LLM as context.

grounded answer

The catch: the precise reranker is too slow to run on millions. So stage 1 narrows to ~100, and stage 2 only reranks those. That split — recall first, precision second — is industry standard (Google, Bing, Meta).

Dense Retrieval (costlier but more accurate): Cross-Encoder



$$\mathbf{z} = \text{BERT}(\text{q}; [\text{SEP}]; \text{d})[\text{CLS}]$$

$$\text{score}(q, d) = \text{softmax}(\mathbf{U}(\mathbf{z}))$$

Use a single encoder to jointly encode query and document and finetune to produce a relevance score with a linear layer over the CLS token. This is too compute-expensive to use except in rescoring

AI4ICPS IIT Kharagpur

Who does each stage: bi- vs cross-encoder

Two jobs, not two rivals — one retrieves, one reranks

Bi-encoder — retrieve

query

document

↓ *encoded separately* ↓

[vector]

[vector]

compare = similarity

- Doc vectors are pre-computed offline
- Search = fast similarity → scales to millions
- Approximate: pulls top ~100 candidates

Cross-encoder — rerank

query + document (together)

↓ *read jointly by one model* ↓

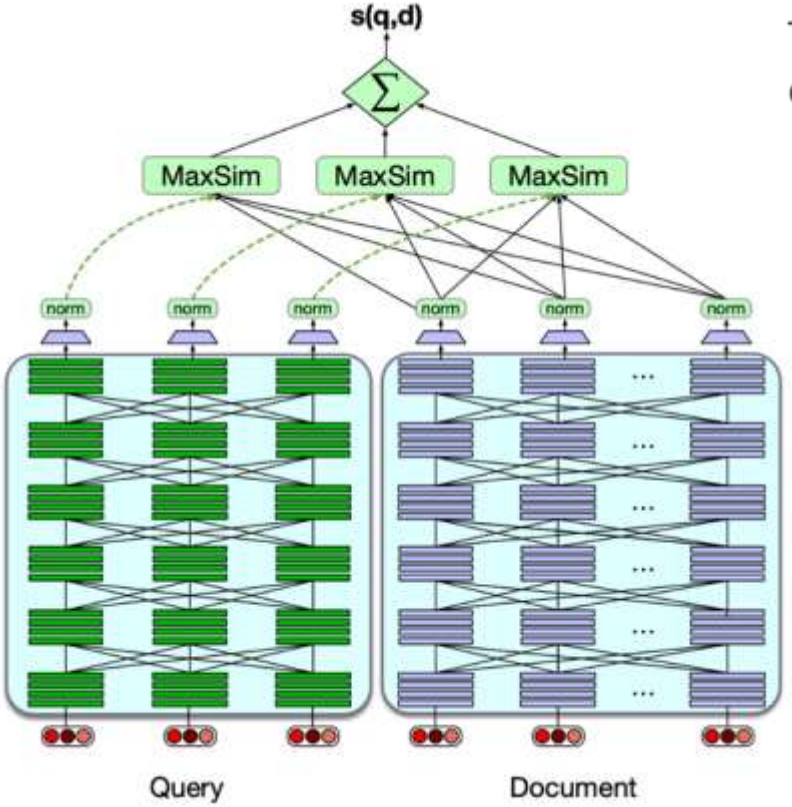
score

sees word-by-word interaction

- Runs at query time on each pair
- Far more accurate relevance score
- Too slow for millions → only the ~100

Separate = fast & scalable (retrieve). Together = precise but costly (rerank).

ColBERT



Precompute document but store each word vector

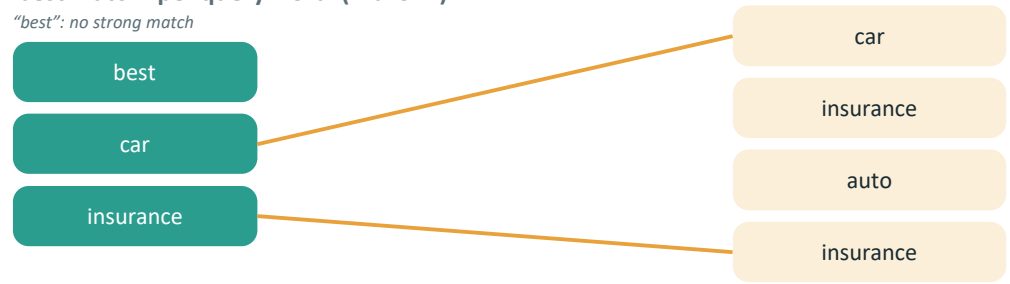
Then do maxsim between document and query words

$$\text{score}(q,d) = \sum_{i=1}^{N} \max_{j=1}^{m} \mathbf{E}_{q_i} \cdot \mathbf{E}_{d_j}$$

ColBERT: a middle ground (nice to know)

Between one-vector-per-doc and full cross-encoder

Store a vector per word, not per document. For each query word, find its best-matching word in the doc, then add those best matches up.



Where it sits

- ✓ **More precise** than one-vector-per-doc — keeps word detail
- ✓ **Much cheaper** than a cross-encoder at scale
- ! **Costs storage** many vectors per document

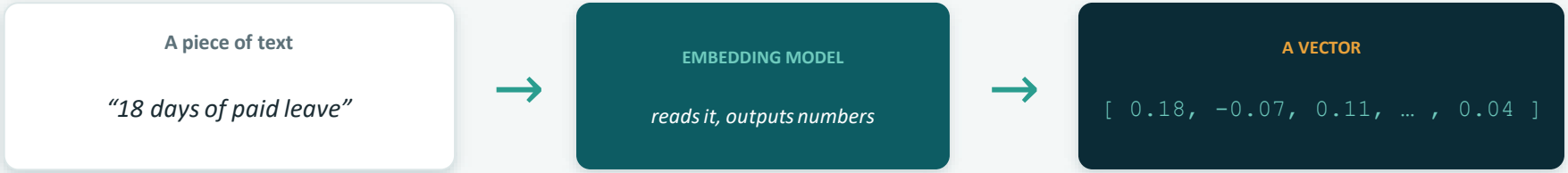
AI4ICPS IIT Kharagpur

Retrieval-Augmented-Generation: Searching the Vector Database





The big idea: turn meaning into numbers through embeddings

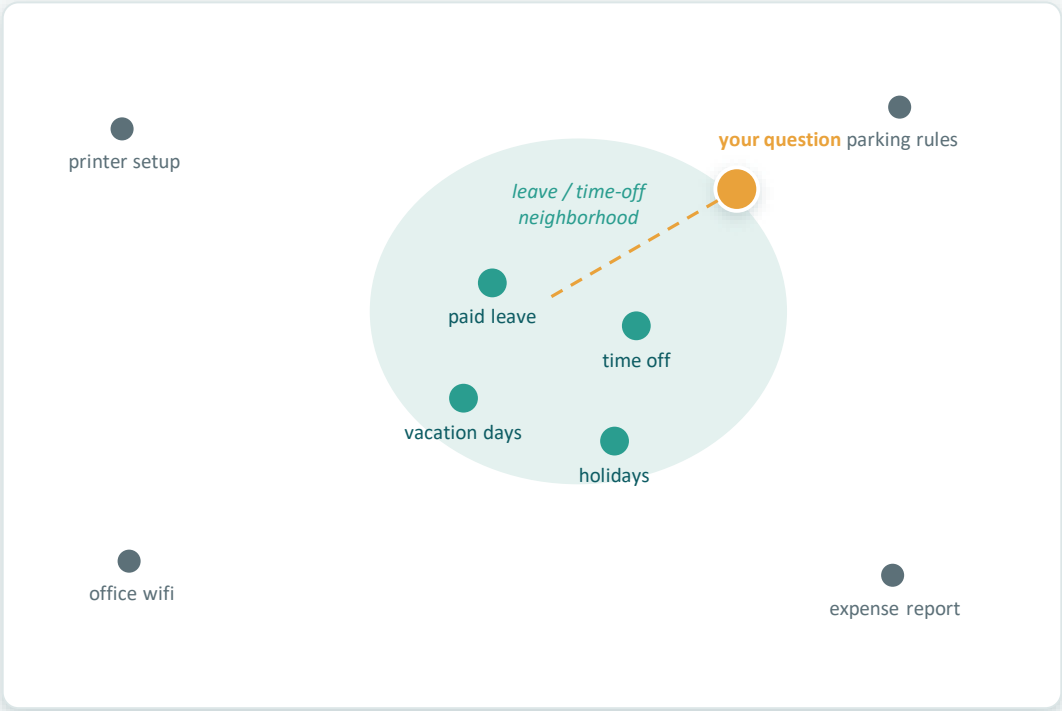


Think of a vector as a “fingerprint of meaning”

- **Same list, every time.** The model always turns the same text into the same numbers.
- **Similar meaning → similar numbers.** Two passages about time off get nearly identical fingerprints.
- **It’s just data now.** A computer can compare meaning by comparing lists of numbers.

Similar meanings sit close together

Picture every vector as a dot on a "map of meaning"



Search = "find the nearest dots."

1. Your question becomes a dot too — using the same model.
2. The database returns the closest dots by meaning.
3. Different words, same neighborhood: "time off" lands right next to "paid leave."

So… what is a vector database?

A place that stores these meaning-vectors and can find the **closest ones** — **fast** — even across millions of records.

DB

Stores three things

Each record keeps the vector, the original text, and metadata — together.

◎

Finds by meaning

Give it a query; it returns the nearest chunks by meaning, ranked.

⤢

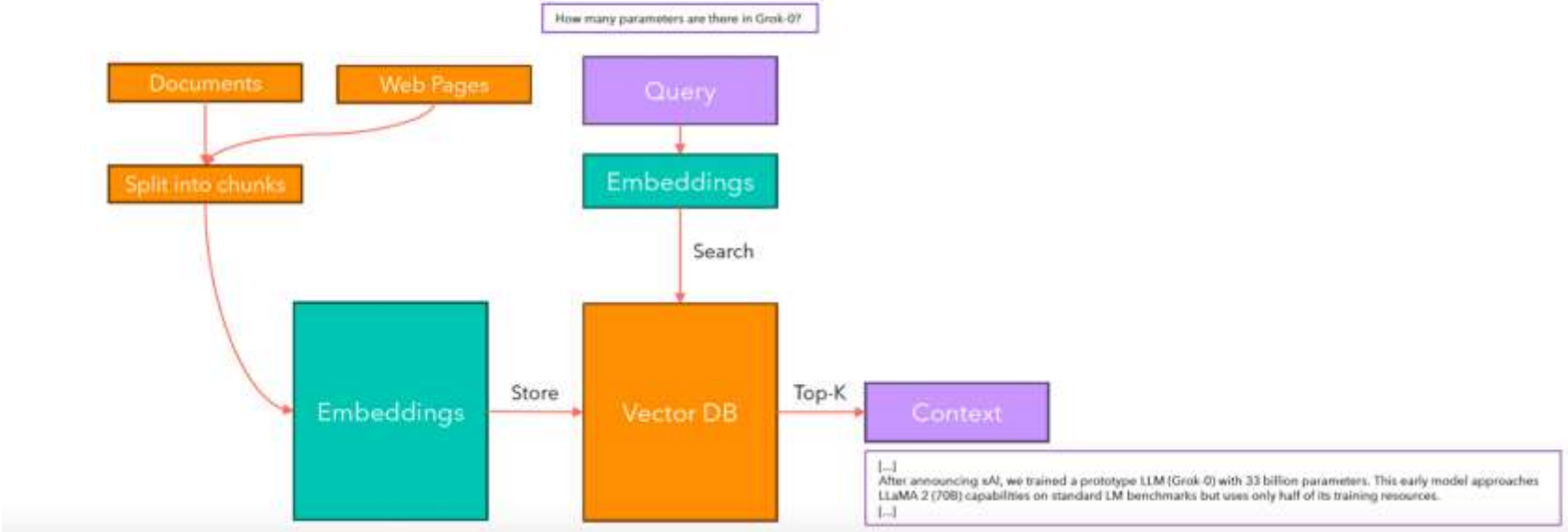
Scales up

Built to search millions of vectors in milliseconds — the naive way can’t.

AI4CPS IIT Kharagpur

Vector Database: Introduction

A vector database stores vectors of fixed dimensions (called embeddings) such that we can then query the database to find all the embeddings that are closest (most similar) to a given query vector using a distance metric, which is usually the cosine similarity, but we can also use the Euclidean distance. The database uses a variant of the KNN (K Nearest Neighbor) algorithm or another similarity search algorithm. Vector DBs are also used for finding similar songs (e.g. Spotify), images (e.g. Google Images) or products (e.g. Amazon).



Source: https://github.com/hkproj/retrieval-augmented-generation-notes

AI4ICPS IIT Kharagpur

One row in the vector store: what actually gets saved

Each chunk becomes a single record — the vector for the math, the original text for the LLM, and metadata to filter & cite.

chunk_id: `doc_2024_hr_policy — p12 — c03`

vector

```
[ 0.0182, -0.0731,  0.1145,  0.0329, -0.0918, … 763 more… ,  0.0447 ]
```

768 float32 numbers — this is what similarity search compares against

text

"Employees accrue 18 days of paid leave annually, credited monthly. Unused leave up to 30 days carries over into the next calendar year."

metadata

`source:` hr_policy_2024.pdf `page:` 12 `section:` "Leave" `date:` 2024-04-01 `lang:` en

Vector → find

Query is embedded the same way; nearest vectors = most similar meaning.

Text → answer

The matched chunk's text goes into the prompt — the LLM reads words, not numbers.

Metadata → filter & cite

Pre-filter (source = HR) and cite "hr_policy_2024.pdf, p.12".

Answers Q88, Q90, Q140 — what's stored (vector + text + metadata), provenance, and doc/page citation.

AI4ICPS IIT Kharagpur

How RAG actually uses it

The vector DB does one job in the chain — fetch the right chunks

1
User question
"How much time off do I get?"

→

2
Embed it
same model → a query vector

→

3
Search the DB
find the nearest chunks by meaning

→

4
Get top chunks
the actual text + its source

→

5
LLM answers
reads the chunks, replies with a cite

The database's only job is step 3: fetch the right chunks. If the wrong chunks come back, no clever prompt can save the answer.

Semantic search in action

Different words in, right passage out

Question: *“How much time off do I get each year?”*

↓ *the DB matches by meaning, not words*

TOP MATCH · hr_policy_2024.pdf · p.12

“Employees accrue 18 days of paid leave annually, credited monthly. Unused leave up to 30 days carries over into the next calendar year.”

↓ *the LLM reads the text and answers*

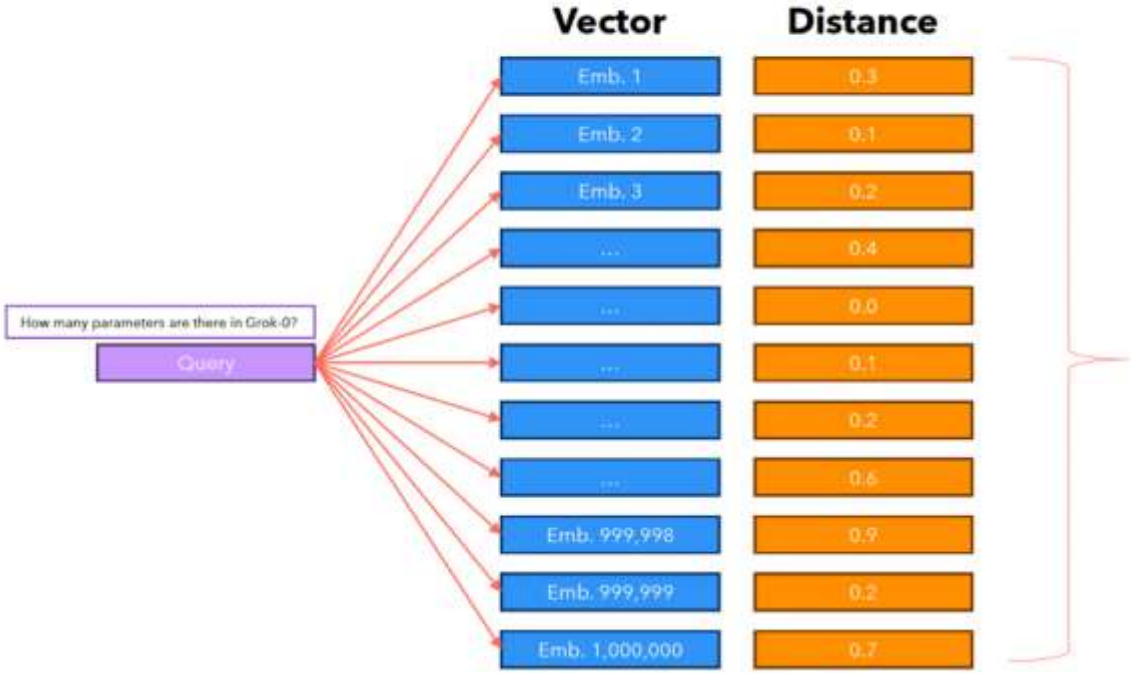
Answer: You get 18 days of paid leave a year, credited monthly, and can carry over up to 30 unused days.
(hr_policy_2024.pdf, p.12)

Why Efficient Vector DB is required for RAG?

1. A critical aspect that powers the capabilities of Retrieval-Augmented Generation models is the vector database that stores the embeddings for fast semantic search during the initial retrieval stage.
2. In order for RAG models to scale to immense corpora containing billions of text passages, efficiently indexing and querying vector representations is crucial.
3. Examples - Weaviate, Chroma, FAISS, Vespa, or Pinecone

KNN: A naive approach

Imagine we want to search for the query in our database: a simple way would be comparing the query with all the vectors, sorting them by distance, and keeping the top K.





If there are N embedding vectors and each has D dimensions, the computational complexity is in the order of **O(N*D), too slow!**

Source: https://github.com/hkproj/retrieval-augmented-generation-notes

AI4ICPS IIT Kharagpur

Finding the nearest — without checking everything

Comparing your query to all 10 million vectors is too slow. So databases take shortcuts.

≡ **Like a library:** you don’t read every book to find one topic — you walk to the right section, then scan that shelf.

Group & skip

Sort similar vectors into clusters. Only search the few clusters near your query — skip the rest.

(a.k.a. IVF)

Compress

Shrink each vector into a tiny code so millions fit in memory and comparisons get cheaper.

(a.k.a. PQ)

Shortcut graph

Link vectors to their neighbors, then “hop” toward the answer — a few jumps instead of a full scan.

(a.k.a. HNSW)

You almost never build these yourself — you pick an “index” by name and the database does all of this for you.

The one trade-off you'll actually tune

Every fast-search trick has the same dial



Turn it DOWN

Check fewer clusters / candidates.

✓ Faster ✕ May miss the true best match

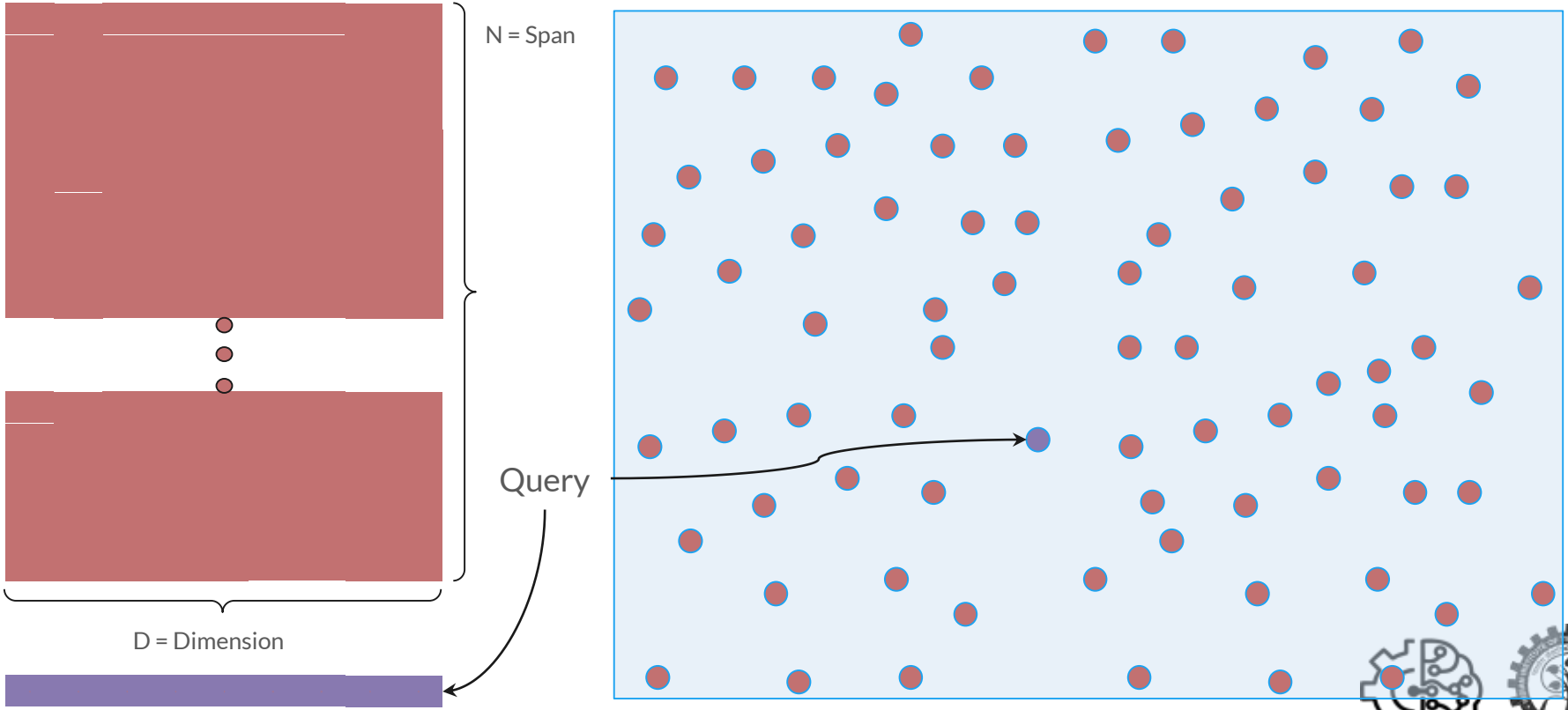
Turn it UP

Check more clusters / candidates.

✓ More accurate ✕ Slower

Practical rule: start fast (dial low). If you notice the right chunks aren't coming back, turn it up until recall is good enough.

IVF: Inverted file for vector Database



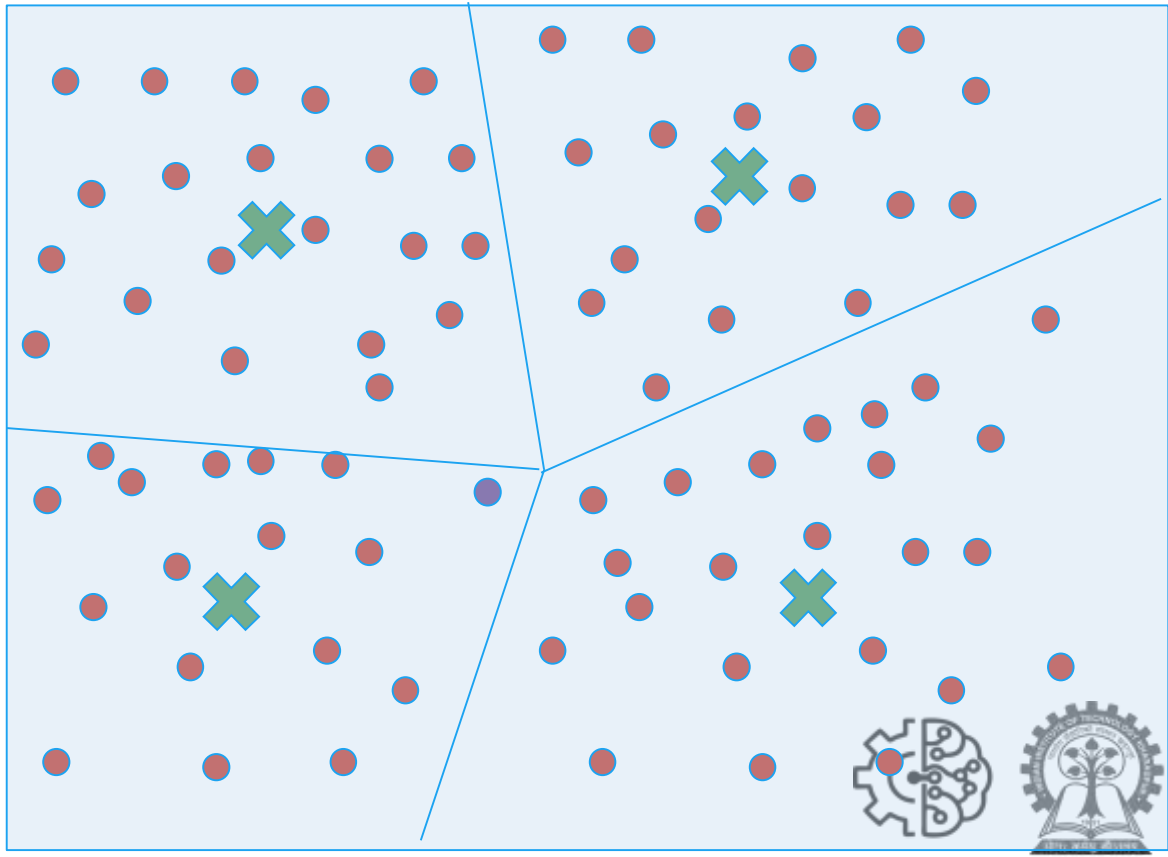
https://www.youtube.com/watch?v=BMYBwbkbVec&list=PLIUOU7oqGTLhlWpTz4NnuT3FekouIVlqc&index=9

AI4ICPS IIT Kharagpur

IVF: Inverted file for vector Database

Phase 1: Indexing

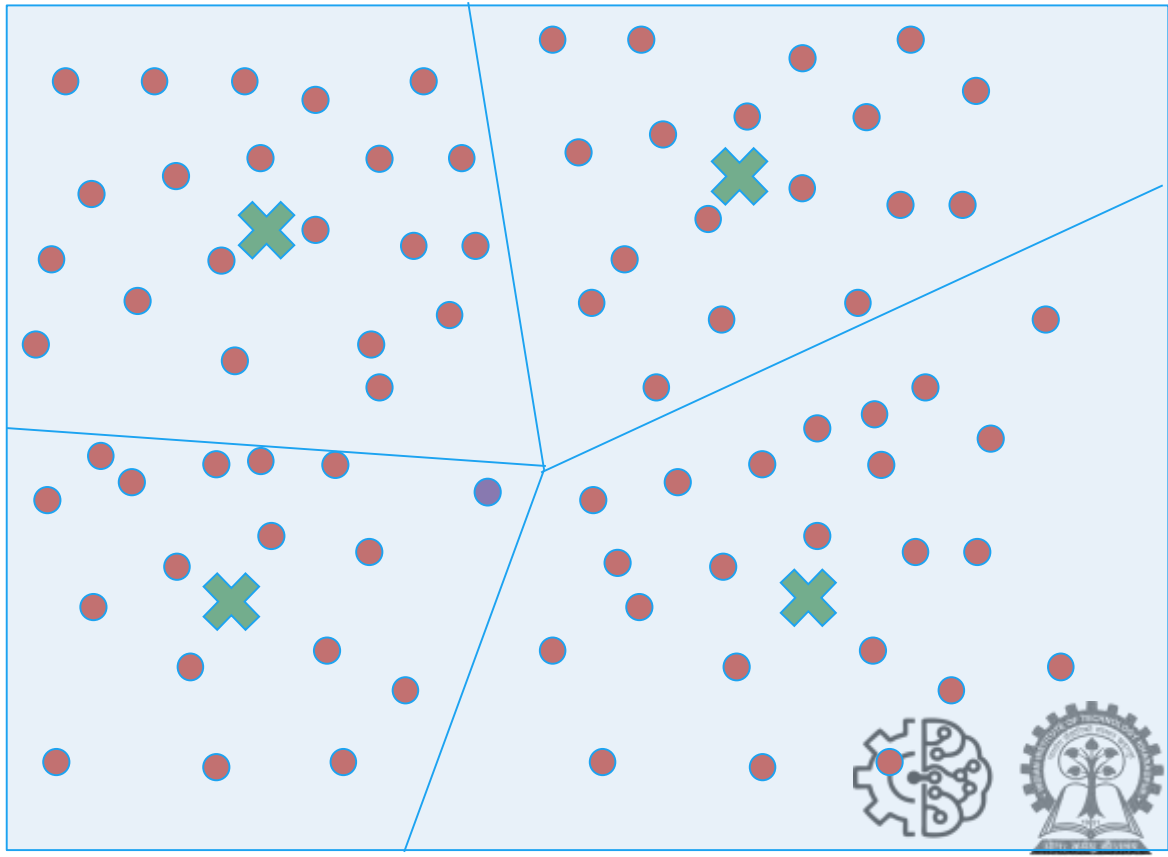
- **Train:** Run **K-Means** on the dataset to identify k representative centroids.
- **Partition:** Assign every vector to its nearest centroid (geometrically forming **Voronoi cells**).
- **Store:** Organize vectors into "buckets" (Inverted Lists) labeled by their centroid ID.



IVF: Inverted file for vector Database

Phase 2: Retrieval (The Search)

- **Coarse Search:** Compare the query **only** against the k centroids to find the closest match.
- **Fine Search:** Scan **only** the vectors inside that specific bucket (ignoring the rest)
- **nprobe** : #clusters (centroids) the algorithm evaluates during an ANN search (*=1 by default*)
- **Outcome:** Reduces search scope from O(N) to approximately O(N/k)



IVF: how much work do you actually save?

QUESTION

N = 1,000,000 vectors, nlist = √N buckets, nprobe = 1. Roughly how many vectors get scanned vs. brute force? Then: what does nprobe = 10 cost?

ANSWER

nlist = 1000 centroids + ~1000 vectors in the one bucket ≈ 2,000 comparisons vs. 1,000,000 — about a 500× reduction.

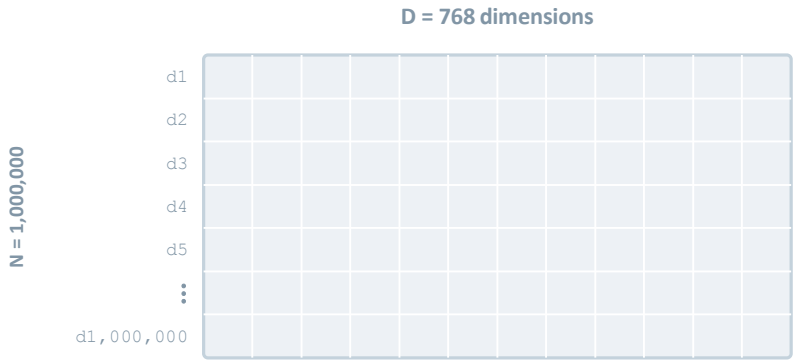
nprobe = 10 → 1000 + ~10,000 ≈ 11,000: still ~90× faster, with markedly higher recall.

Product Quantization

We have **N = 1,000,000** database vectors, d_1 through $d_{1,000,000}$.

Each is a **D = 768**-dimensional vector of floats — one row of the matrix. Storing them all raw is the problem PQ solves.

Over the next slides we compress this whole set: split each row, cluster each segment, and keep only the cluster IDs.



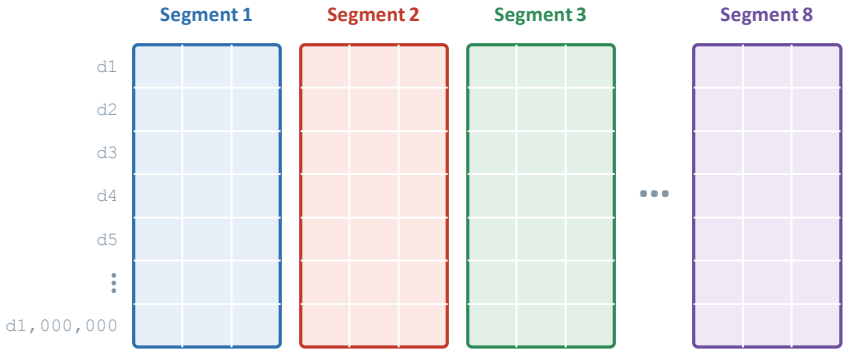
N = 1,000,000 vectors · each D = 768 dims
Rows d_1 … $d_{1,000,000}$, each holding 768 floats.

AI4ICPS IIT Kharagpur

Split every vector into 8 segments

Cut the D = 768 columns into **m = 8** equal slices of 96 dims each. Every vector now has 8 sub-vectors — one per segment — compressed independently.

- **Segment 1**
- **Segment 2**
- **Segment 3**
- *… through …*
- **Segment 8**



AI4ICPS IIT Kharagpur

STEP 2 OF 3

Cluster each segment → 8 codebooks of 256 centres

Run **K-means** inside each segment across all one million sub-vectors. With **K = 256**, every segment gets 256 cluster centres — its codebook C_i.

C1 — Segment 1

256 centres · ids 0–255

c1,0	id 0
c1,1	id 1
⋮	
c1,255	id 255

C2 — Segment 2

256 centres · ids 0–255

c2,0	id 0
c2,1	id 1
⋮	
c2,255	id 255

C3 — Segment 3

256 centres · ids 0–255

c3,0	id 0
c3,1	id 1
⋮	
c3,255	id 255

…

C8 — Segment 8

256 centres · ids 0–255

c8,0	id 0
c8,1	id 1
⋮	
c8,255	id 255

Why 256? an id in 0–255 fits in exactly one byte — so each segment costs 1 byte, and a whole vector costs just 8.

AI4ICPS IIT Kharagpur

Encode: each vector becomes 8 cluster IDs

Find the nearest centre in every segment and keep its id. The saved table is the compressed database — columns and rows abbreviated with …

Vector	Seg 1	Seg 2	Seg 3	…	Seg 8
d1	12	5	200	…	88
d2	42	201	7	…	99
⋮	⋮	⋮	⋮	⋮	⋮
d1,000,000	255	60	9	…	120

Read a row

d_2 → [42, 201, 7, …, 99]

one id per segment — 8 bytes in total.

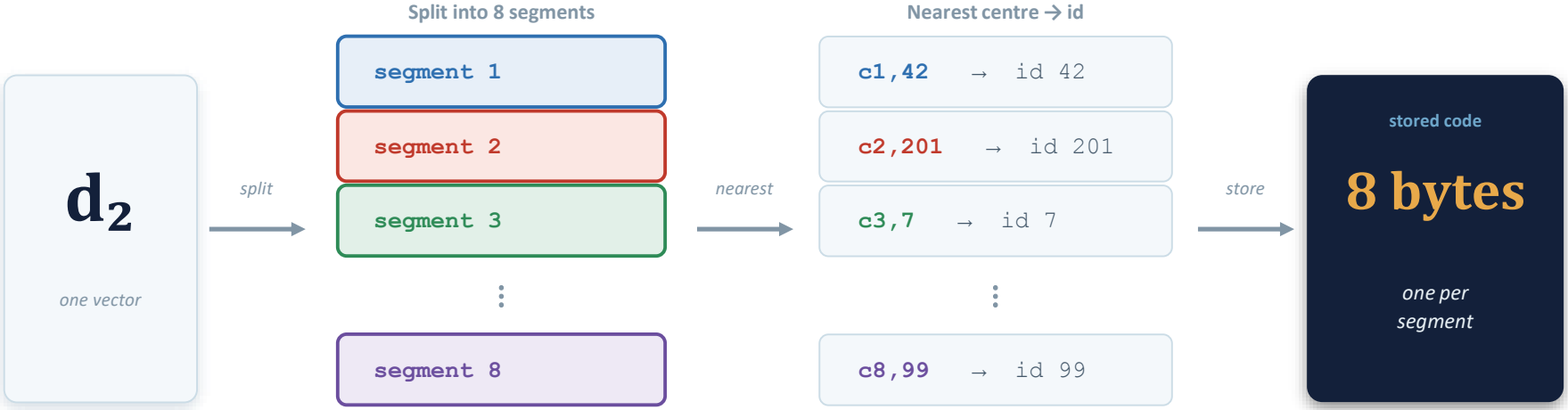
Discarded: the original float sub-vectors are dropped. Only the 8 IDs per vector are kept.

AI4ICPS IIT Kharagpur

ZOOM IN

Following one vector, d_2, across its 8 segments

Split into 8 sub-vectors, match each to its nearest of 256 centres, keep the id. Segments abbreviated with …



d_2 → [42, 201, 7, …, 99] 8 one-byte IDs replace 768 floats. Every vector encodes the same way.

AI4ICPS IIT Kharagpur

THE PAYOFF

One million vectors, a fraction of the memory

BEFORE

3 GB

1M × 768-dim
as 32-bit floats

→

AFTER

8 MB

1M × 8 one-byte
segment IDs

384×

smaller · ~99.7% saved

Size per vector

`raw` = D × 4 bytes (3 KB at D=768)

`coded` = m bytes = 8 bytes

m = 8 segments, K = 256 → one byte each. Independent of D.

Why it matters at a million

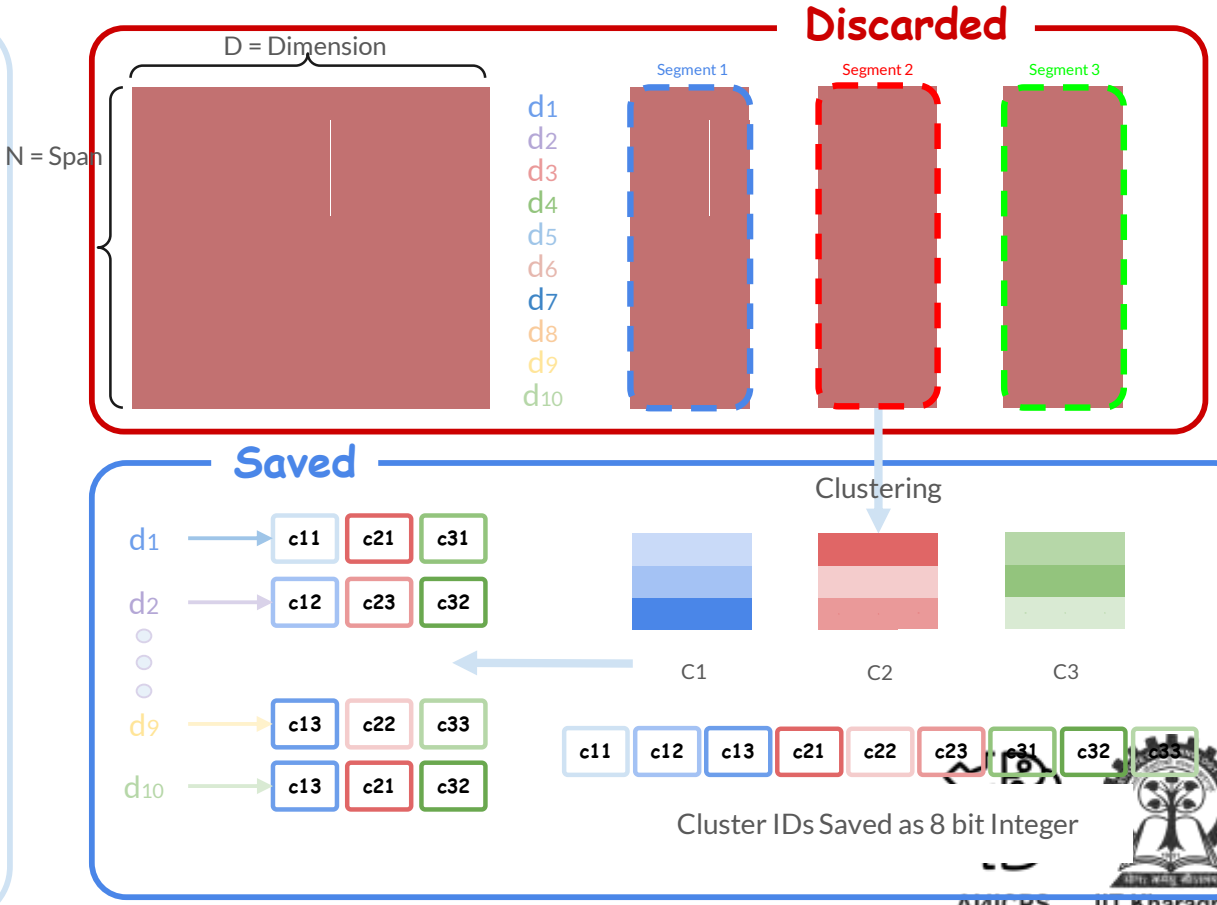
3 GB of raw vectors strains memory and won't scale to billions; 8 MB fits easily — the index stays in RAM and search stays fast.

More segments (m) → slightly larger codes, better accuracy.

PQ: Product Quantisation (Indexing)

Indexing (Compression Phase)

Split (Sub-vectors): Divide the high-dimensional original vector (e.g., 128D) into m smaller sub-vectors (e.g., 8 chunks of 16D).
Train (Codebooks): Run K-Means independently on each sub-space to find K (eg. 256) representative centroids (the "Codebook").
Encode: Replace each sub-vector with the **ID** (0-255) of its nearest centroid.
Result: High-precision floats are converted into small byte codes, reducing memory usage by 95%+ (e.g., 4KB →64 bytes).



QUERY TIME

Search: do the multiplications once, then just add

Brute force compares the query to every stored vector — a full distance each, a mountain of multiplications. A **distance lookup table** turns almost all of that into lookups and adds.

BRUTE FORCE (NO PQ)

Distance to every vector

Compute the full distance from the query to all 1,000,000 stored vectors — 768 multiplies each.

768M

multiplications per query

→

ONCE PER QUERY

Build one distance table

Measure the query against the 256 centroids in each of the 8 segments, then reuse it for all.

≈197K

multiplications, computed once

→

PER VECTOR

Look up + add

Each vector's 8 IDs index 8 cells; sum them for its approximate distance. No multiplies.

8 lookups, 7 additions

Lookup, adds per vector (× 1M)

Asymmetric Distance Computation (ADC): the heavy multiplies run once (≈197K), not a million times (768M) — about **3,900× fewer multiplications**. The million-vector scan is then only lookups and adds, because the query stays full-precision while the stored vectors are compressed IDs.

PQ search: compute once, then just add

The database vectors are stored as **centroid IDs**, not floats. So distance becomes a **table lookup + addition** — no float math per vector.

1 Once per query: build the distance table

Split the query into m sub-vectors. For each, measure its distance to that sub-space's centroids.

	c1	c2	c3	c4
q_1	0.9	**0.2**	0.7	0.5
q_2	0.4	0.6	**0.1**	0.8
q_3	0.3	0.7	0.9	**0.2**

m × k table — tiny, reused for every vector

2 Per database vector: look up + add

A stored vector is just its centroid IDs, e.g. **[c2, c3, c4]**. Read those cells and sum them.

$q_1 \to c2$ **0.2** + $q_2 \to c3$ **0.1** + $q_3 \to c4$ **0.2** = **0.5**

approx. distance — 3 lookups, 2 adds

Why it's fast — Asymmetric Distance Computation (ADC)

"Asymmetric" = the query stays full-precision while documents are compressed. The costly query–centroid math happens **once**; scanning a million vectors is then just lookups and additions — no per-vector float distance.

Contrast with the indexing slide: there you built the codebooks; here you reuse them to score fast.

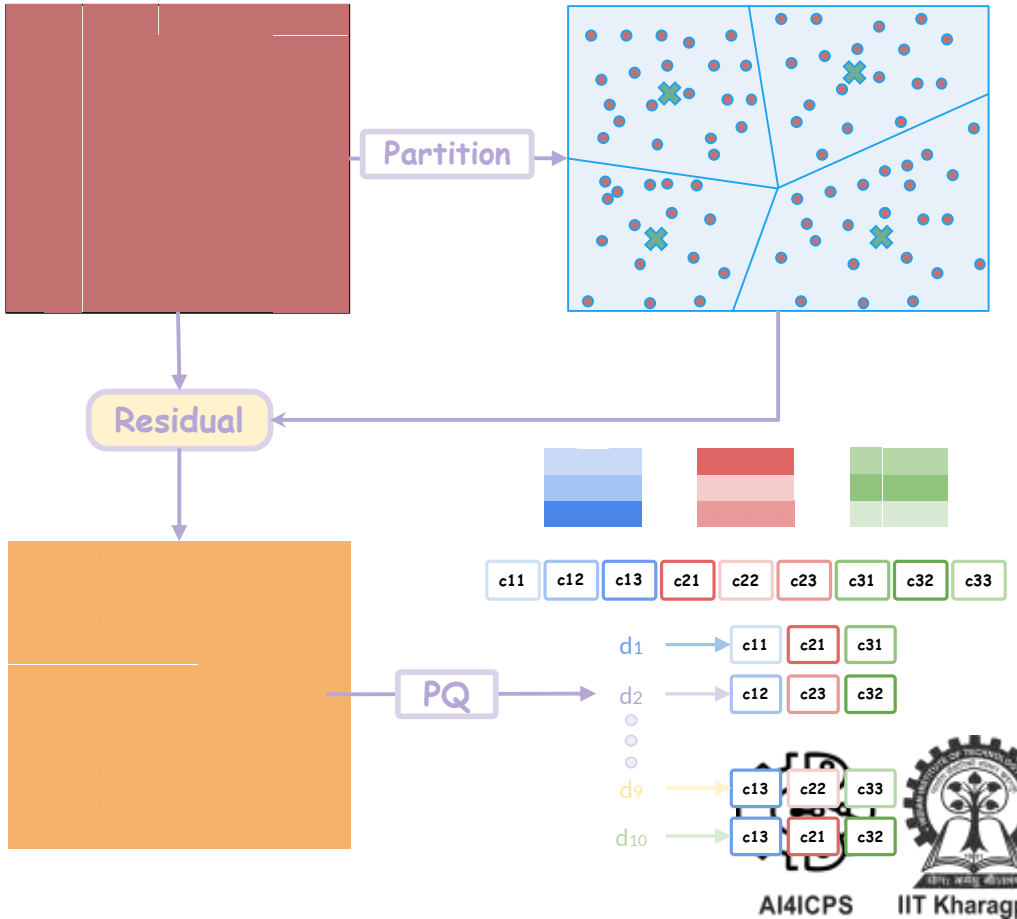
IVF + PQ

IVF and PQ each solve two different problems:

1. **IVF** solves **"Too many vectors to check"** (by clustering).
2. **PQ** solves **"Vectors are too big to store"** (by compression).

IVF+PQ Indexing (Construction)

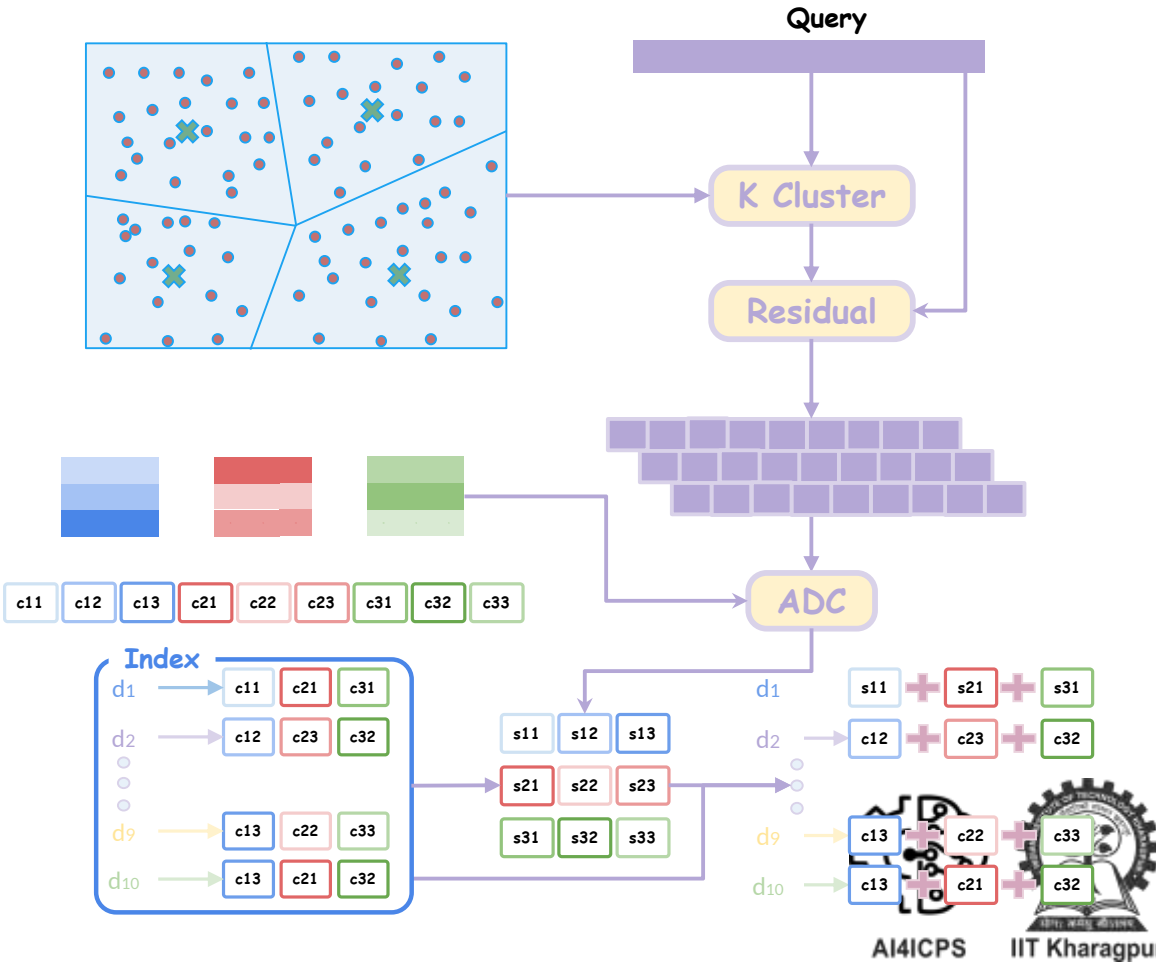
- **1. Coarse (IVF):** Run K-Means (e.g., 1024 clusters) on the dataset to find coarse centroids.
- **2.** Assign each vector to its nearest centroid, then calculate the difference vector (Residual = Vector - Centroid).
- **3. Fine (PQ):** Split the **residual vector** into sub-vectors and train the PQ codebooks on these residuals.
- **4.** Save the PQ codes (byte IDs) into the specific Inverted List for that centroid.



IVF + PQ

IVF+PQ Search (Retrieval)

- 1. **Coarse (IVF):** Compare the query against the coarse centroids to find the nprobe closest buckets (e.g., top 10).
- 2. For each selected bucket, shift the query vector relative to that bucket's centroid (Queryshifted=Query−Centroid).
- 3. **Build Tables (ADC):** Construct the PQ distance lookup tables using these shifted queries.
- 4. **Scan:** Iterate through the PQ codes in the selected buckets, summing values from the lookup table to approximate distance.



A practical superpower: filtering

Search only the documents that matter — before doing any vector math

People ask scoped questions

“In the HR policy, how many sick days…”

“According to the 2024 handbook…”

“In the engineering docs, how do I…”

The scope is a filter — so use it.

Metadata (source, date, section) lets you narrow the search first.

One line narrows the search

```
where = { "source": "hr_policy.pdf" }
```

✓ **Higher precision** Only HR passages can come back — no stray matches from other docs.

✓ **Faster** A smaller candidate set means less to search through.

✓ **Fewer surprises** The answer stays on-topic and easy to cite.

Two tools you'll meet: ChromaDB vs FAISS

A ready-made database vs a raw search engine you assemble yourself

C ChromaDB

a database

- Stores text + metadata + vectors together — automatically
- Saves to disk (persists) on its own
- Filters by metadata out of the box
- A working search service in ~10 lines
- Best up to a few million vectors

F FAISS

a library

- Extremely fast, highly tunable — scales to billions
- Full control over index type & compression
- NOT a database: no text, metadata, or persistence
- You keep the id → text/metadata map yourself
- Lose that map = "numbers but no documents" bug

ChromaDB: a database wrapped around vector search

FAISS is a **library** that finds nearest vectors. ChromaDB is the **database** that wraps it with everything a RAG app actually needs.

The raw-search problem

A bare similarity search returns **numbers**:

```
"vector #4471 is closest"
```

… but the LLM needs the **text**, and you need the **source** to filter & cite. With FAISS you track that ID→text map yourself — the classic *"numbers but no documents"* bug.

Chroma stores all three together

Vector	for the similarity math
Text	the actual chunk the LLM reads
Metadata	source, page, date — to filter & cite

The feature raw search can't do: metadata filtering

```
where={"source": "policy.docx"}
```

Pre-filter before the search → higher precision AND a smaller, faster candidate set.

The trade-off

Under the hood: HNSW (one index, CPU, single node). Convenience now, a ceiling later — graduate to FAISS for billions or fine index control.

In one line: take an ANN library like FAISS, wrap it in a real database — text, metadata, persistence, filtering included.

AI4ICPS IIT Kharagpur

Some more finer details about RAG

WHERE THIS FITS

Two steps happen before retrieval



This deck lives in these two steps — the ones that quietly make or break every answer.

AI4ICPS IIT Kharagpur

DOCUMENT PROCESSING

Garbage in, garbage out

Retrieval quality is capped by how cleanly you pull text — and structure — out of a document before chunking.

Extraction

Pulls the raw characters off the page. You get text — but no idea what any of it is.

Parsing

Goes further: labels each block — heading, table row, caption — and keeps how they relate.

For RAG, structure is as valuable as the text. Knowing a number sits in a table row — not loose prose — is what lets the model use it correctly.

DOCUMENT PROCESSING

Know your source format

Each format hands you text differently. Recognising which one you have tells you how hard extraction will be.

PDF

Built for looks, not structure.

- Multi-column text bleeds together
- Scanned PDFs are images → need OCR
- Hardest to get clean

DOCX

Structured XML under the hood.

- Headings, lists, tables already labelled
- Relationships preserved
- Easiest, most reliable to parse

HTML

Rich tags, but noisy.

- Semantic tags: h1, p, table
- ~90% is nav, footers, ads
- Strip boilerplate first

DOCUMENT PROCESSING

Match the tool to the document

Don't reach for a heavy parser to solve a problem you don't have. Pick by what the document actually is.

DOCUMENT	REACH FOR	WHY
Plain-prose PDF	**pypdf**	Fast and light; no structure to preserve
Table-heavy PDF	**pdfplumber / Camelot**	Keeps rows & columns instead of word-salad
Scanned PDF	**OCR (tesseract / VLM)**	No text layer exists — nothing to extract
Complex layout	**Unstructured / docling**	Recovers reading order in multi-column pages
HTML	**BeautifulSoup / trafilatura**	Targeted for your site, or heuristic for the open web

THE #1 TRAP

Probe before you parse

Image-only PDFs return empty strings silently. Check for a real text layer first, or you'll ship empty chunks to production.

probe.py

```python
import pypdf

def has_text_layer(path, sample=3):
    r = pypdf.PdfReader(path)
    return any((pg.extract_text() or "").strip()
               for pg in r.pages[:sample])

# route on the result, never assume
if has_text_layer("file.pdf"):
    text = parse_with_pypdf(path)
else:
    text = run_ocr(path)    # scanned document
```

What's happening

Silent failure

pypdf returns "" on image-only PDFs — no error.

Probe first

Check a few pages for any real text.

Then route

Text layer → parser · none → OCR.

Rule of thumb: empty chunks are worse than slow ones. A silent miss ships bad answers.

STRUCTURED DATA

Query tables — don't embed them

Relational data and big tables are answered by generating SQL, not by cramming rows into a vector store.

Why embedding rows fails

Aggregations — SUM, AVG, COUNT, GROUP BY aren't similarity search

"Top 5 by revenue" — is a computation, not a nearest-neighbour lookup

Exact filters — date ranges and IDs are exactly what SQL is for

The pattern: text-to-SQL

- **Retrieve the schema —** table + column descriptions, not the rows
- **LLM writes the SQL —** generated per question, not pre-listed
- **Run it, then phrase —** feed the rows back for a natural answer

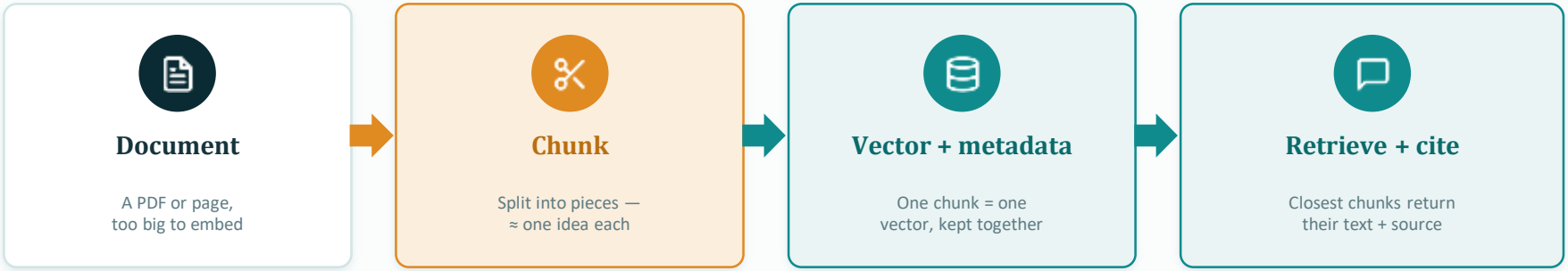
Hybrid apps route: *prose → vector RAG, numbers & tables → SQL. A data-lake chatbot usually needs both.*

AI4ICPS IIT Kharagpur

CHUNKING

The chunk is the unit of retrieval

RAG never retrieves whole documents — it retrieves chunks. So how you split decides what can ever be found.



The goal: each chunk holds one self-contained idea — small enough to be precise, large enough to stand alone. Metadata (source, page) rides along, so answers can cite.

CHUNKING

Four ways to split

From simplest to smartest. Most projects start at sliding window and only move up if retrieval needs it.

Fixed-size

Split every N tokens.

Simplest baseline — but cuts straight through ideas.

DEFAULT

Sliding window

Fixed-size + 10–20% overlap.

The sane default — catches facts on boundaries.

Structure-aware

Split on headers, clauses, functions.

The doc shows the seams — precise and citable.

Semantic

Cut where meaning shifts.

Best quality — but embeds every sentence to ingest.

AI4ICPS IIT Kharagpur

Two more boundary-based splitters

Between blunt fixed-size and clever recursive sit two options that respect the writing's own units.

Sentence-based

Split at sentence endings.

- Keeps grammar intact and readable.
- But chunk sizes vary wildly — some tiny, some huge.

Paragraph-based

Split on paragraph breaks.

- Preserves larger, self-contained thoughts.
- But a dense academic paragraph can overflow the window.

Either way: respect natural boundaries, but always cap chunk size so one run-on section can't blow past the model's limit.

CHUNKING · THE SANE DEFAULT

Recursive chunking, step by step

Don't cut blindly by length. Try the biggest natural boundary first, and only fall back when a chunk is still too big.

The fallback ladder

1. **Paragraphs** `split on \n\n`
2. **Sentences** `then on . or \n`
3. **Phrases** `then on ,`
4. **Words** `then on spaces`
5. **Characters** `last resort — brute cut`

Stop as soon as a chunk fits your size limit.

Why not just fixed-size?

A blind cut at 1024 chars leaves *"…convolutional architectures are supe"* — embedding broken meaning wrecks retrieval.

Overlap still matters

Even after clean splits, a fact that spans two chunks needs a small slice of the previous chunk (~10–20%) carried over, or you lose the continuity across embeddings.

CHUNKING

Sensible defaults

There is no universal best size — but these starting points get you a working baseline fast.

≈512 tokens, 10–20% overlap

A solid starting point for general prose.

Count tokens, not characters

Use the embedding model's own tokenizer.

Chunk on natural boundaries

Sentences and paragraphs beat arbitrary limits.

Match size to content

Small for Q&A facts; larger for narrative or code.

Always evaluate & iterate

Test retrieval on real questions; tune from there.

The overlap dial

0% overlap

Facts on a boundary get sliced in two.

~20% overlap

Catches them cheaply. The knee of the curve.

50%+ overlap

Near-duplicate chunks — double the store, little gain.

AI4ICPS IIT Kharagpur

DOES IT ACTUALLY WORK?

Evaluate retrieval and generation separately

You're grading two systems at once. Test each — a great generator can't rescue bad retrieval.

Retrieval — did we find it?

Precision@K — how much noise in the top K

Recall — did we get all the relevant chunks

MRR — how high is the first right hit

NDCG — relevance and rank position

Generation — is the answer good?

Faithfulness — claims backed by the context

Answer relevancy — does it address the question

Context adherence — uses only what was retrieved

Completeness — covers the relevant info

RAGAS automates this — reference-free, using an LLM as judge. Four core metrics — context precision, context recall, faithfulness, answer relevancy — roll into one score you can track over time.

AI4ICPS IIT Kharagpur

EVALUATION, DEEPER

How LLM-as-a-judge really works

Never ask a model for a vibe score. Break the answer into atomic claims, check each against a reference, then count.

1 · Decompose

answer → list of standalone claims

2 · Verify each

claim vs context: supported? + reason

3 · Aggregate

score = supported ÷ total (0–1)

Worked example — faithfulness

Answer: *"Marie Curie won two Nobel Prizes and was born in 1867."*

- "won two Nobel Prizes" — *in context*
- "a scientist" — *in context*
- "born in 1867" — *not in context*

Score = 2 ÷ 3 = 0.67

Make the judge trustworthy

- **Pin the model + temperature 0 —** so scores are repeatable run to run
- **Give a rubric, demand a reason —** a verdict with a reason, not a lone number
- **Mind judge bias —** length, position, and self-preference
- **Calibrate on human labels —** check agreement on a small sample

Why it works: many small yes/no checks are more consistent — and auditable — than one holistic 1–10 guess.

UNDER THE HOOD

The prompts RAGAS actually runs

Faithfulness is two LLM calls: pull the claims out of the answer, then judge each one against the retrieved context.

1 · Extract claims (LLM call)

```
Break the answer into standalone
statements. Resolve pronouns so each
one stands on its own. One per line.

question: {question}
answer:   {answer}
```

2 · Verify each claim (LLM call)

```
For each statement, decide if it can be
inferred from the context.
Return a verdict (1 = yes, 0 = no) and a
one-line reason for each.

context:    {retrieved_chunks}
statements: {statements}
```

faithfulness = **Σ verdicts ÷ number of claims** · the other three follow the same decompose-and-check idea:

- **Answer relevancy —** the LLM writes the questions this answer would answer → cosine-compare to the real one
- **Context precision —** judge each retrieved chunk: was it useful for the answer? (reward hits ranked high)
- **Context recall —** judge each ground-truth sentence: is it supported by the retrieved context?

Retrieval Evaluation: Rank-based measures

- Binary relevance
  - Precision@K (P@K)
  - Mean Average Precision (MAP)
  - Mean Reciprocal Rank (MRR)
- Multiple levels of relevance
  - Normalized Discounted Cumulative Gain (NDCG)

Precision@K

- Set a rank threshold K
- Compute % relevant in top K
- Ignores documents ranked lower than K
- Ex:
  - Prec@3 of 2/3
  - Prec@4 of 2/4
  - Prec@5 of 3/5



- In similar fashion we have Recall@K

Mean Average Precision

- Consider rank position of each ***relevant*** doc
  - $K_1, K_2, \ldots K_R$
- Compute Precision@K for each K= $K_1, K_2, \ldots K_R$
- Average precision = average of P@K
- Ex:  has AvgPrec of $\frac{1}{3} \cdot \left(\frac{1}{1} + \frac{2}{3} + \frac{3}{5} \right) \approx 0.76$
- MAP is Average Precision across multiple queries/rankings

Average Precision



= the relevant documents

Ranking #1



Recall	0.17	0.17	0.33	0.5	0.67	0.83	0.83	0.83	0.83	1.0
Precision	1.0	0.5	0.67	0.75	0.8	0.83	0.71	0.63	0.56	0.6

Ranking #2



Recall	0.0	0.17	0.17	0.17	0.33	0.5	0.67	0.67	0.83	1.0
Precision	0.0	0.5	0.33	0.25	0.4	0.5	0.57	0.5	0.56	0.6

$$\text{Ranking \#1: } (1.0 + 0.67 + 0.75 + 0.8 + 0.83 + 0.6)/6 = 0.78$$

$$\text{Ranking \#2: } (0.5 + 0.4 + 0.5 + 0.57 + 0.56 + 0.6)/6 = 0.52$$

AI4ICPS IIT Kharagpur

MAP


= relevant documents for query 1

Ranking #1



Recall	0.2	0.2	0.4	0.4	0.4	0.6	0.6	0.6	0.8	1.0
Precision	1.0	0.5	0.67	0.5	0.4	0.5	0.43	0.38	0.44	0.5


= relevant documents for query 2

Ranking #2



Recall	0.0	0.33	0.33	0.33	0.67	0.67	1.0	1.0	1.0	1.0
Precision	0.0	0.5	0.33	0.25	0.4	0.33	0.43	0.38	0.33	0.3

$$average\ precision\ query\ 1 = (1.0 + 0.67 + 0.5 + 0.44 + 0.5)/5 = 0.62$$
$$average\ precision\ query\ 2 = (0.5 + 0.4 + 0.43)/3 = 0.44$$

$$mean\ average\ precision = (0.62 + 0.44)/2 = 0.53$$

AI4ICPS IIT Kharagpur

Production Rag Systems (What's different?)



Query Rewriting

If the RAG system is a chatbot, the preceding simple RAG implementation would likely struggle with the search step if a question is too verbose, or to refer to context in previous messages in the conversation.

> User Question: "We have an essay due tomorrow. We have to write about some animal. I love penguins. I could write about them. But I could also write about dolphins. Are they animals? Maybe. Let's do dolphins. Where do they live for example?"

This should actually be rewritten into a query like:

> Query: "Where do dolphins live"

This rewriting behavior can be done through a prompt (or through an API call).

Query Routing

An additional enhancement is to give the model the ability to search multiple data sources. We can, for example, specify for the model that

If it gets a question about HR, it should search the company's HR information system (e.g., Notion) but
If the question is about customer data, that it should search the customer relationship management (CRM) (e.g., Salesforce).

Closer to Agentic RAG: *Data sources can be abstracted into tools*

Thank You!!



